DESTINATION DURHAM · COMPUTER SCIENCE TASTER

Durham University

How a Computer *Really* Works

From a single switch to a running program in two hours

Dr Carlton Shepherd · Durham University

cs.gl/riscv-web-emulator/

About Me

- Dr Carlton Shepherd (Assistant Professor of Computer Science).
- I teach Computer Systems (1st year) and Networked Systems (2nd year).
- Areas of expertise: cyber security, device security (or: breaking and fixing things in a way that appears professionally respectable).

So… what actually IS a computer?

By the end of this session, you will hopefully have a better understanding.

THE NEXT TWO HOURS

Where we're going

1

Just switches

Binary: everything is on/off

2

The ladder

Layers from electrons to apps

3

Meet the CPU

Registers, memory, fetch–execute

4

Make it add

Step through real instructions

Break

10-15 minutes

5

Poke the machine

Change memory & registers by hand

6

C → machine code

Watch a language become steps

7

Draw with code

Paint pixels into memory

PART 1 · FIRST PRINCIPLES

It's all just switches

A computer can't store a “7”, a letter, or a photo. It only has switches: on or off – or, zeros and ones representing those states. Everything else is built upon this.

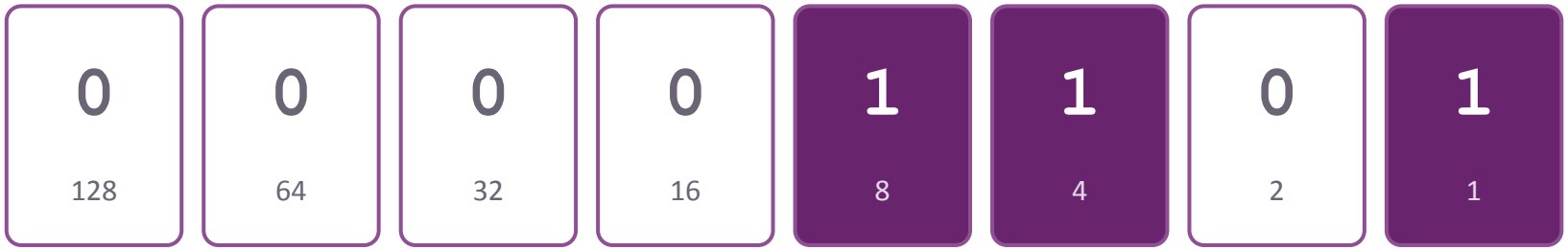
1

One switch = one bit. Eight switches = one byte.

Every switch is worth **double** the one to its right. Turn some on, add up their values — that's a number in **binary**.

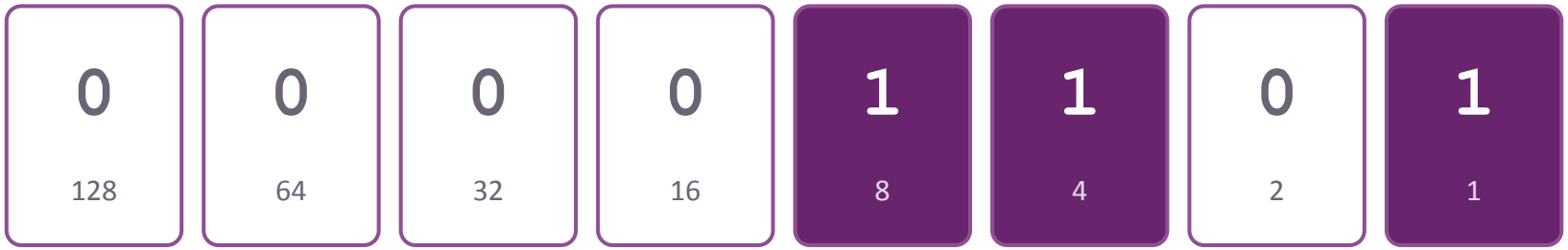
One switch = one bit. Eight switches = one byte.

Every switch is worth **double** the one to its right. Turn some on, add up their values — that's a number in **binary**.



One switch = one bit. Eight switches = one byte.

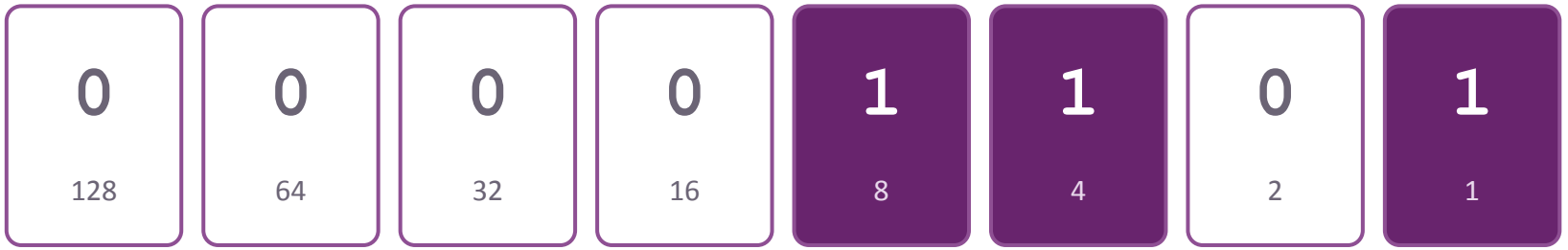
Every switch is worth **double** the one to its right. Turn some on, add up their values — that's a number in **binary**.



8 + 4 + 1 = **13**

One switch = one bit. Eight switches = one byte.

Every switch is worth **double** the one to its right. Turn some on, add up their values — that's a number in **binary**.



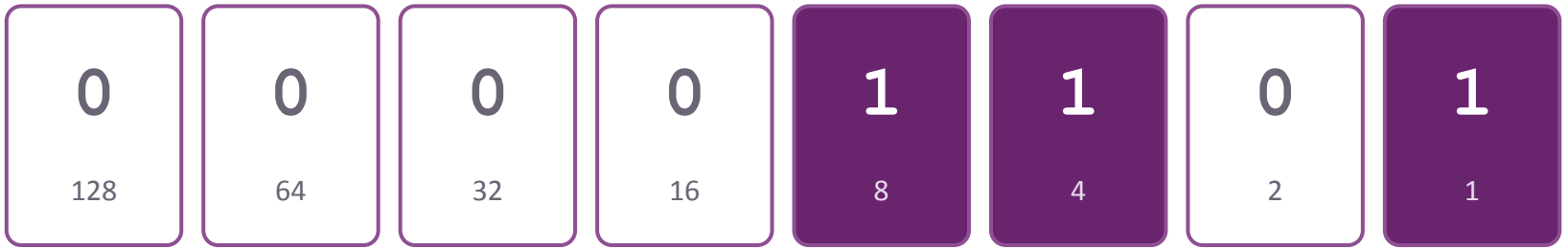
8 + 4 + 1 = **13**

256

different values fit in one byte — the numbers 0 to 255. That's the ceiling of 8 switches.

One switch = one bit. Eight switches = one byte.

Every switch is worth **double** the one to its right. Turn some on, add up their values — that's a number in **binary**.



8 + 4 + 1 = **13**

256

different values fit in one byte — the numbers 0 to 255. That's the ceiling of 8 switches.

The computer never "understands" anything; it just moves 0s and 1s insanely fast.

✋ Try it together

1 Make 13

Which switches turn do you turn on/off?

2 Make 42

Trickier.

 Go as big as you can

If we put two bytes side by side — 16 switches — what is the biggest number you can make?

Even letters are secretly numbers

There's a rule — called **ASCII** — that maps each letter to a number. The computer stores the number; we read it as a letter.

A	B	C	H	i
= 65	= 66	= 67	= 72	= 105

Even letters are secretly numbers

There's a rule — called **ASCII** — that maps each letter to a number. The computer stores the number; we read it as a letter.

A	B	C	H	i
= 65	= 66	= 67	= 72	= 105

65 = A = 01000001. The same bits are a number AND a letter; context decides which. That one idea is why the same machine can do maths, write essays, and show photos.

PART 2 · THE BIG PICTURE

The ladder of abstraction

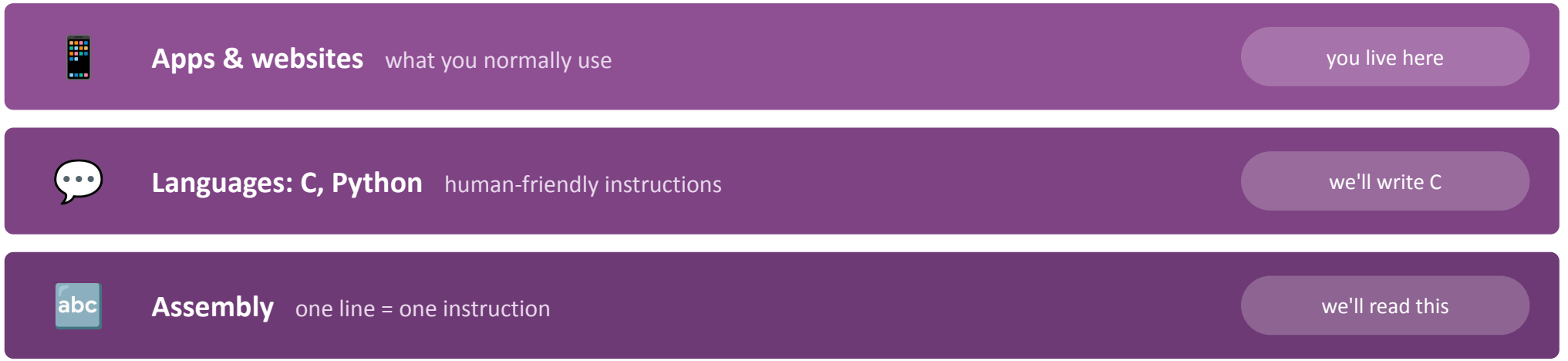
Tapping an app sits at the top of a tall ladder. Let’s go down it.

2

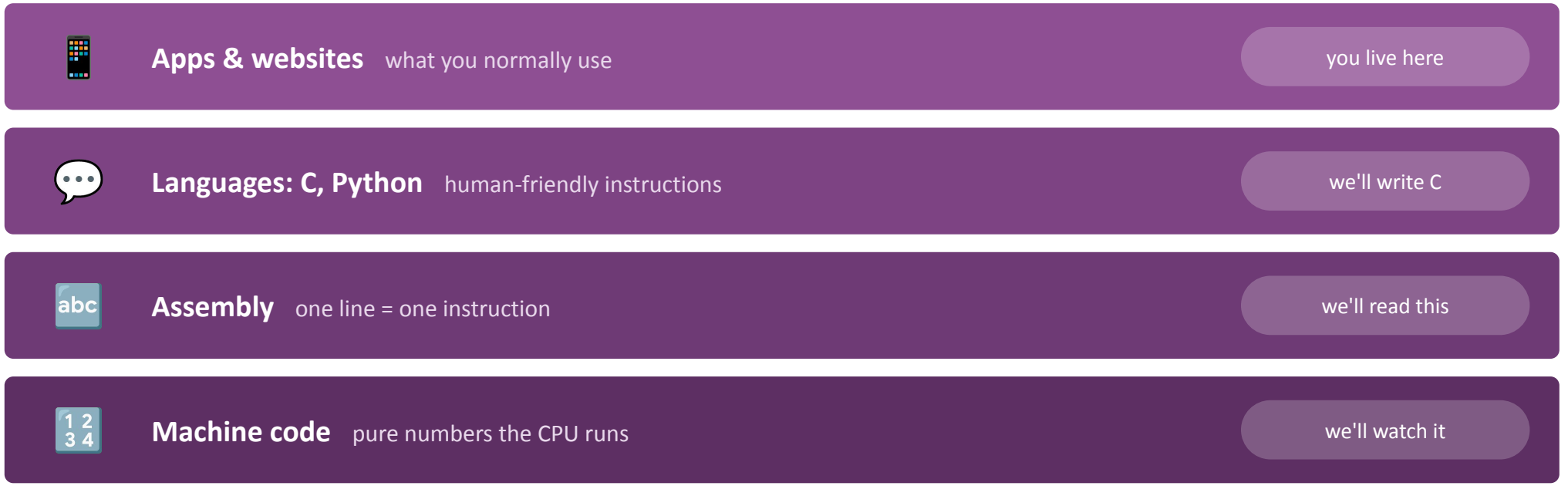
Six rungs from electricity to apps

Six rungs from electricity to apps

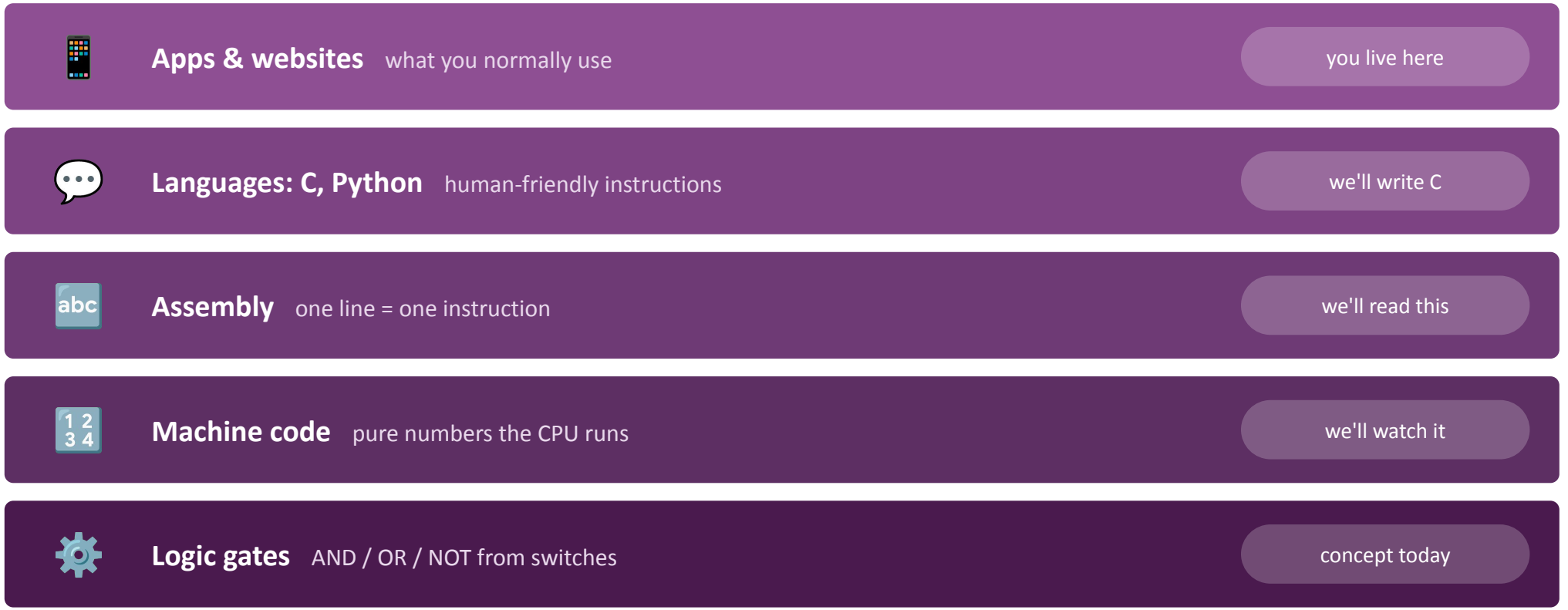
Six rungs from electricity to apps



Six rungs from electricity to apps



Six rungs from electricity to apps



Six rungs from electricity to apps

📱	**Apps & websites** what you normally use	you live here
💬	**Languages: C, Python** human-friendly instructions	we'll write C
🔤	**Assembly** one line = one instruction	we'll read this
🔢	**Machine code** pure numbers the CPU runs	we'll watch it
⚙️	**Logic gates** AND / OR / NOT from switches	concept today
⚡	**Transistors & electricity** physical on/off switches	the bottom rung

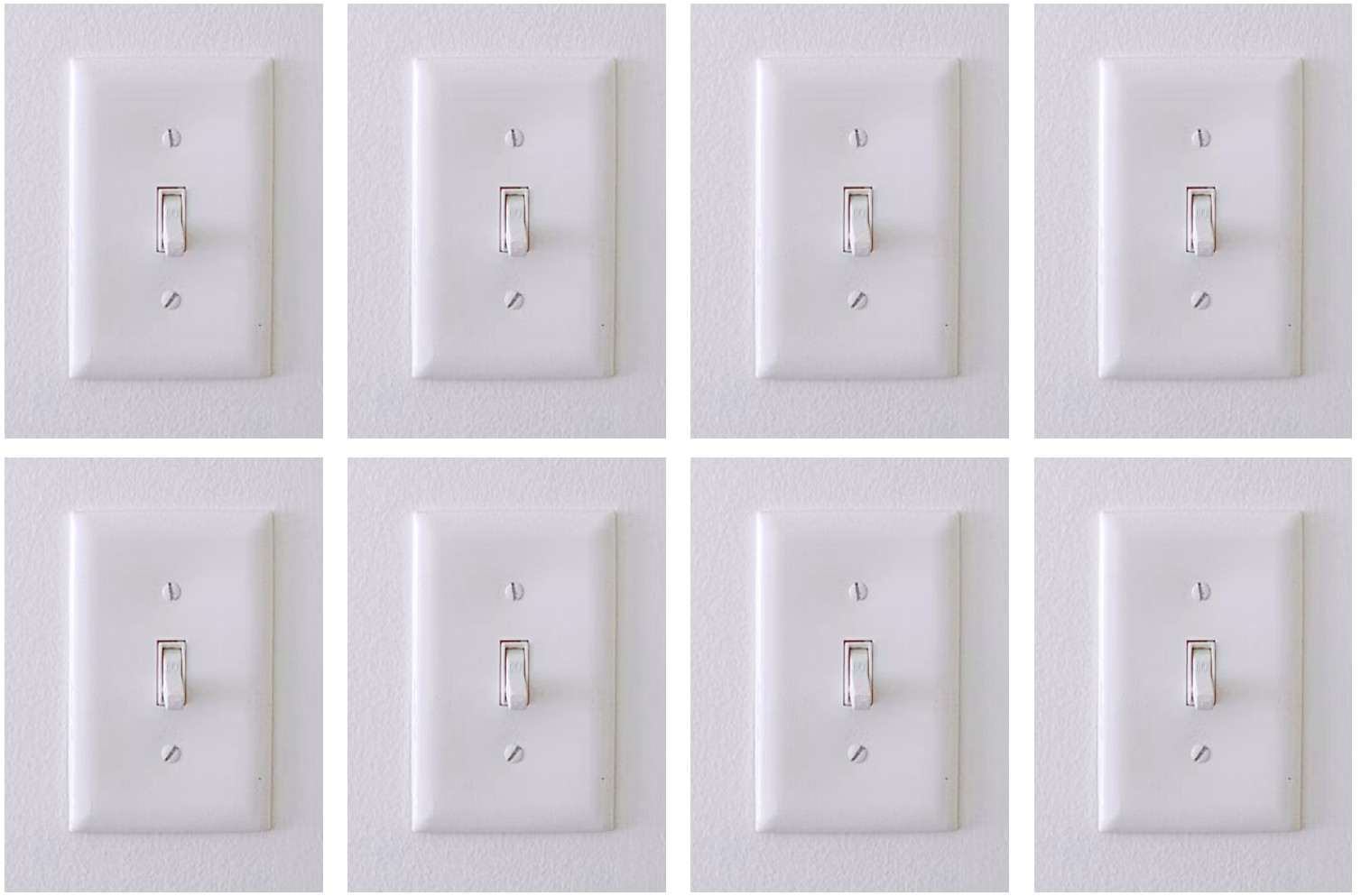
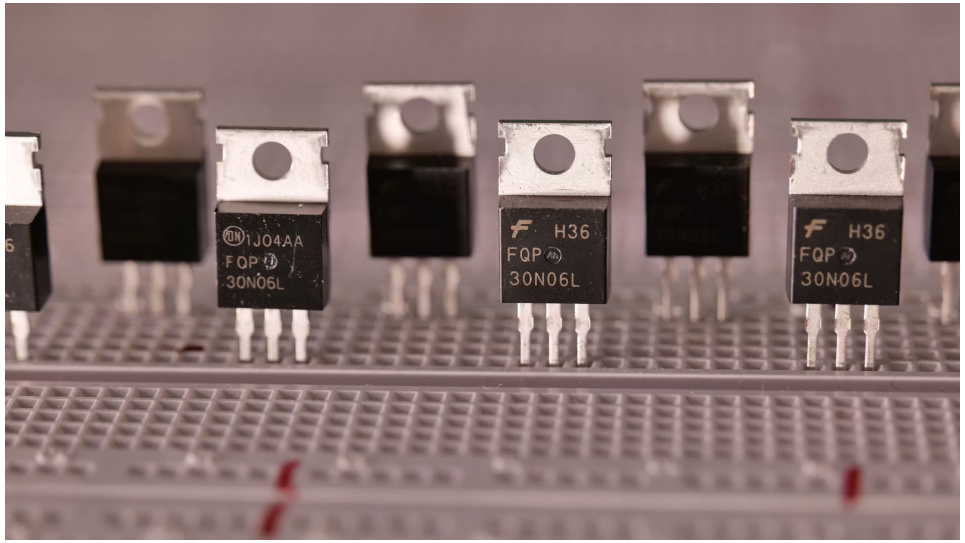
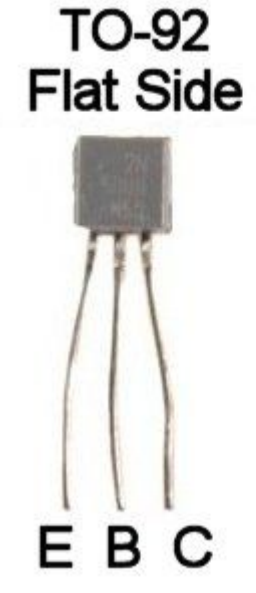


Image source, Unsplash.com





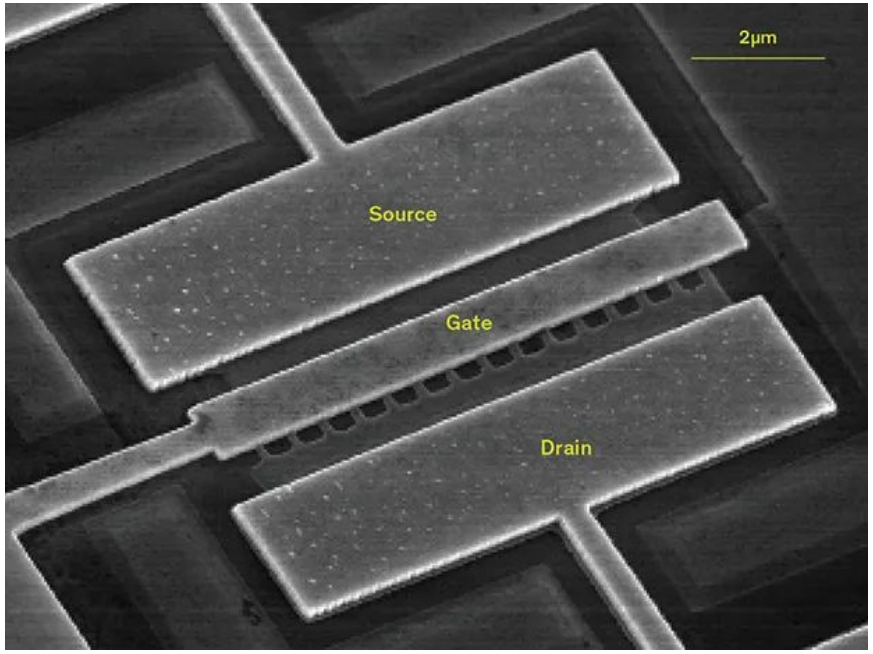


Image sources: IEEE Spectrum, Unsplash.com, EDN News

PART 3 · INSIDE THE BOX

The CPU: fast, and gloriously stupid

A processor does one boring thing billions of times a second. Let's see exactly what.

3

The fetch–execute cycle

1

Fetch

Grab the next instruction. The program counter (pc) remembers which one.

2

Decode

Work out what it means: add? load? jump?

3

Execute

Do it — usually move or add one number.

4

Repeat

Move pc on. Start again. Forever.

That's the whole job. Billions of times a second, it just fetches, decodes, executes, repeats. No cleverness — only speed.

Two places the CPU keeps numbers

Registers: the desk

About 32 tiny slots *inside* the CPU. Lightning fast, almost no room; only what you're working on right now.

```
pc  — which instruction is next
a0  — a number being worked on
t0  — scratch / temporary
sp  — top of the stack
```

Two places the CPU keeps numbers

Registers: the desk

About 32 tiny slots *inside* the CPU. Lightning fast, almost no room; only what you're working on right now.

```
pc   — which instruction is next
a0   — a number being worked on
t0   — scratch / temporary
sp   — top of the stack
```

Memory: the filing cabinet

Millions of numbered slots (addresses). Bigger, a little slower. The CPU carries numbers between cabinet and desk all day.

```
.text   — the instructions
.data   — set-up values
stack   — working notes
0x10000 — the 64×64 screen
```

PART 3 · LIVE TOUR

Meet the RV32 Workbench

C code

Write here in a friendly language

Assembly

The CPU's real instructions

Registers

The desk — pc, a0, t0 …

Console

What the program prints

Memory

The filing cabinet

64×64 screen

Pixels drawn from memory

https://cs.gl/riscv-web-emulator/

1 Make it add 2 + 2 (10 mins)

Goal: *watch one instruction run and change a number in front of you.*

1. In the C code box, make the program add 2 + 2 and print it.
2. Look at the Assembly panel: find a line with add or addi.
3. Set the speed to 8/second (or press Step).
4. Watch pc tick forward each step.
5. Keep going until 4 appears in a register, then the Console.

```
int main() {
  print_int(2 + 2);
  return 0;
}
```

✏ **Predict first:** How many Step presses before the 4 appears? Write your guess, then count.

🏋 **Stretch:** Change 2 + 2 to 7 * 6. Set the ISA to RV32IM. Can you spot a mul instruction appear?

2 Muck around with the machine by hand (10m)

Goal: *prove registers and memory are just numbers you can change.*

1. Reset, then Step a few times so registers hold values.
2. Click a register value (try t0) and type 99.
3. Do the same to a slot in the Memory .data area.
4. Press Run and watch your change ripple through.

```
// nothing to type —
// just click values
// and edit them!

t0  =  99
```

🏋️ **Stretch:** Add a line in the Assembly panel, e.g. addi a0, a0, 10. Reset, run — did the answer shift by 10?

3 Watch C become machine code (10m)

Goal: *see one friendly line explode into many tiny CPU steps.*

1. Enter the loop that adds up 1 to 10.
2. On paper, work out 1+2+…+10 first.
3. Run it: did you get 55?
4. Count the Assembly lines: 3 lines of C became many.
5. Slow to 8/sec: watch one register count 1,2,3…

```
int main() {
  int total = 0;
  for (int i=1;i<=10; i++)
    total += i;
  print_int(total);
}
```

✏ **Predict first:** What is 1+2+3+…+10 ? Work it out before you run.

🏋 **Stretch:** Change i <= 10 to i <= 100. New answer? Then add only the even numbers (start at 2, i = i + 2).

4 Paint pixels straight into memory (10m)

Goal: *draw on a real screen by writing numbers — the payoff.*

1. The 64×64 screen lives in memory from 0x10000.

This is some real computers work. We access devices by a number called its memory address (fancy name: memory-mapped I/O)

2. Each pixel is one byte (0–255) = its colour.
3. Fill the screen red with the loop opposite.
4. Try 0x1C green, 0x03 blue, 0xFF white, 0x00 black.

5. Make bands: screen[i] = (i/8)%2 ? 0xE0 : 0x1C;

```
char *fb  = 0x10000;
for (int i=0;i<64*64; i++)
   fb[i] = 0xE0; // red
while(1){}
```

✏ **Predict first:** You never call “draw”. Why does writing numbers change the screen?

🏋 **Stretch:** Use each pixel's position (row = i/64, col = i%64) to invent a gradient or checkerboard. Show your neighbour.

PART 8 · ZOOM OUT

What you just did

You now know

A computer only stores 0s and 1s. Numbers, text and images are all just patterns of them.

You realised

A CPU is a fast, simple machine: fetch, decode, execute. Moving numbers between registers and memory.

You saw that

Code is translated into tiny instructions, and you can read and even write them (but we rarely do this — its too cumbersome!).

In two hours you went from a single switch to running real code and drew a picture by hand-placing numbers in memory. That's the actual machine, with the magic removed.

A computer is not magic.

It's a very fast machine following very simple rules on 0s and 1s. Now you've seen the rules.

Questions?

Dr Carlton Shepherd · Durham University